

HW4

Sri Srinivasan

July 2025

Problem 1

(a)

- Parameter sharing: Convolutional layers use the same filter weights across different spatial positions. This reduces the number of parameters compared to fully connected layers and improves generalization.
- Spatial locality: Convolutions use the spatial structure of image data by connecting each output unit to only a local part of the input. This makes them more efficient.

(b)

We are given:

Input signal: $[1, 4, 0, -2, 3]$, Output after convolution: $[-2, 2, 11]$

Let the filter be $[f_1, f_2, f_3]$.

We find the dot products.

$$1 \cdot f_1 + 4 \cdot f_2 + 0 \cdot f_3 = -2 \quad (1)$$

$$4 \cdot f_1 + 0 \cdot f_2 + (-2) \cdot f_3 = 2 \quad (2)$$

$$0 \cdot f_1 + (-2) \cdot f_2 + 3 \cdot f_3 = 11 \quad (3)$$

From equation (1): $1 \cdot f_1 + 4 \cdot f_2 = -2$

From equation (2): $4 \cdot f_1 - 2 \cdot f_3 = 2$

From equation (3): $-2 \cdot f_2 + 3 \cdot f_3 = 11$

Now we solve this system. From (1): $f_1 = -2 - 4f_2$

Substitute into (2):

$$4(-2 - 4f_2) - 2f_3 = 2 \Rightarrow -8 - 16f_2 - 2f_3 = 2 \Rightarrow -16f_2 - 2f_3 = 10 \quad (4)$$

From (3): $-2f_2 + 3f_3 = 11 \Rightarrow 3f_3 = 11 + 2f_2 \Rightarrow f_3 = \frac{11+2f_2}{3}$

Substitute into (4):

$$-16f_2 - 2 \cdot \frac{11+2f_2}{3} = 10 \Rightarrow -16f_2 - \frac{22+4f_2}{3} = 10 \Rightarrow \frac{-48f_2 - 22 - 4f_2}{3} = 10 \Rightarrow \frac{-52f_2 - 22}{3} = 10$$

$$-52f_2 - 22 = 30 \Rightarrow -52f_2 = 52 \Rightarrow f_2 = -1$$

Then:

$$f_1 = -2 - 4(-1) = -2 + 4 = 2, \quad f_3 = \frac{11 + 2(-1)}{3} = \frac{9}{3} = 3$$

The filter is:

$$\boxed{[2, -1, 3]}$$

(c)

Input:

$$\begin{bmatrix} -1 & 2 \\ 3 & 1 \end{bmatrix}, \quad \text{Filter: } \begin{bmatrix} +1 & -1 \\ 0 & +1 \end{bmatrix}$$

Flipping the filter both horizontally and vertically (since we transpose):

$$\text{Flipped filter: } \begin{bmatrix} +1 & 0 \\ -1 & +1 \end{bmatrix}$$

Each input pixel spreads its scaled filter into a larger output grid. The input is 2×2 and the filter is 2×2 , so the output will be 3×3 . We initialize a 3×3 output matrix with zeros, then for each input value at (i, j) , we add the scaled flipped filter into position $(i : i + 2, j : j + 2)$:

Output:

$$\boxed{\begin{bmatrix} -1 & 0 & 2 \\ 4 & -1 & 1 \\ 0 & 3 & 1 \end{bmatrix}}$$

Problem 2

(a)

We use the output dimension formula for convolution:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

i. Kernel size: 3×3 , stride: 1, padding: 1

$$H_{\text{out}} = \left\lfloor \frac{32 + 2(1) - 3}{1} \right\rfloor + 1 = 32$$

So, output dimensions: $[10, 64, 32, 32]$

ii. Kernel size: 4×4 , stride: 2, padding: 0

$$H_{\text{out}} = \left\lfloor \frac{32 + 0 - 4}{2} \right\rfloor + 1 = 15$$

So, output dimensions: $[10, 64, 15, 15]$

(b)

An example that works is

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

(c)

An example that works is

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

(d.i)

Given:

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad k = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

To compute the 2-dilated convolution:

$$\text{Dilated kernel: } \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Now apply it to M :

$$\text{Region covered: } \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \Rightarrow 1 \cdot 1 + 3 \cdot 1 + 7 \cdot 1 + 9 \cdot 1 = 1 + 3 + 7 + 9 = \boxed{20}$$

(d.ii)

- DilatedConv1: kernel size 3×3 , dilation $d = 1$
Receptive field: 3×3
- DilatedConv2: kernel size 3×3 , dilation $d = 2$
Receptive field: $(3 - 1) \cdot 2 + 1 = 5 \times 5$

Now stacking them:

The receptive field of two stacked dilated layers (DilatedConv1 $\rightarrow$ DilatedConv2) is:

$$(5 - 1) \cdot 1 + 3 = \boxed{7 \times 7}$$

The second layer sees a 5×5 region of the output from the first layer, which itself sees a 3×3 region of the input. Thus, the total receptive field in terms of the original input is 7×7 .

Problem 3

(a)

- Each filter has shape $3 \times 3 \times 3$ (because it spans all 3 input channels)
- There are 4 such filters (one per output channel)

So, the total number of learnable parameters is:

$$\text{Traditional conv parameters} = 3 \times 3 \times 3 \times 4 = \boxed{108}$$

(b)

We have two steps

- One 3×3 filter per input channel (no mixing of channels)
- Input has 3 channels

So the number of parameters is:

$$\text{Depthwise conv parameters} = 3 \times (3 \times 3) = 27$$

- Pointwise (1x1) convolution applied to all depthwise outputs
- Input has 3 depthwise output channels
- Output has 4 channels

Each 1×1 filter takes 3 input channels and produces one output channel.

$$\text{Pointwise conv parameters} = 1 \times 1 \times 3 \times 4 = 12$$

$$\text{Total} = 27 + 12 = \boxed{39}$$

Problem 4

(a)

We begin by expanding the expectation:

$$\mathbb{E} [\|\mathbf{y} - (R \odot X) \tilde{\mathbf{w}}\|_2^2] = \mathbb{E} [\|\mathbf{y} - R(X \tilde{\mathbf{w}})\|_2^2]$$

By linearity of expectation and the properties of Bernoulli(p) variables:

$$= \|\mathbf{y} - pX\tilde{\mathbf{w}}\|_2^2 + p(1-p) \sum_{j=1}^d \|X_j\|_2^2 \tilde{w}_j^2$$

where X_j is the j -th column of X , and $\tilde{\Gamma}$ is the diagonal matrix with $\tilde{\Gamma}_{jj} = \|X_j\|_2^2$.

(b)

From Equation (5), the regularization term is scaled by $p(1-p)\tilde{\Gamma}^2$. To match this to standard ridge regression:

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 + \|\Gamma\mathbf{w}\|_2^2$$

We should transform

$$\mathbf{w} = \tilde{\mathbf{w}}, \quad \text{with } \Gamma = \sqrt{p(1-p)}\tilde{\Gamma}$$

That is, the dropout-trained weight $\tilde{\mathbf{w}}$ can be interpreted as solving a ridge regression with a specific Γ .

(c)

Let $\tilde{\mathbf{w}} = \Gamma\mathbf{w}$ and define the new data matrix $\tilde{X} = X\Gamma^{-1}$. Then:

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2 + \|\Gamma\mathbf{w}\|_2^2 = \|\mathbf{y} - \tilde{X}\tilde{\mathbf{w}}\|_2^2 + \|\tilde{\mathbf{w}}\|_2^2$$

Rescaling the regularization term with a hyperparameter λ gives us ridge regression.

(d)

Since Γ is diagonal with $\Gamma_{jj} \propto \|X_j\|_2$, and $\tilde{X} = X\Gamma^{-1}$, this implies that each column j of $\tilde{X}$ is:

$$\tilde{X}_j = \frac{X_j}{\|X_j\|_2}$$

Therefore, each column is normalized in terms of its ℓ_2 norm, just like in batch normalization (but across the batch dimension).

Problem 5

We are given the expected loss when the input features are elementwise multiplied by a random matrix R , where each entry $R_{ij} \sim \mathcal{N}(\mu, \sigma^2)$. Specifically:

$$\mathcal{L}(\mathbf{w}) = \mathbb{E}_{R_{ij} \sim \mathcal{N}(\mu, \sigma^2)} \left[\|\mathbf{y} - (R \circ X)\mathbf{w}\|_2^2 \right]$$

Let us simplify this expression.

Let $Z = R \circ X$. Then:

$$\begin{aligned} \mathcal{L}(\mathbf{w}) &= \mathbb{E}_R [\|\mathbf{y} - Z\mathbf{w}\|_2^2] = \mathbb{E}_R [(\mathbf{y} - Z\mathbf{w})^\top (\mathbf{y} - Z\mathbf{w})] \\ &= \mathbb{E}_R [\mathbf{y}^\top \mathbf{y} - 2\mathbf{y}^\top Z\mathbf{w} + \mathbf{w}^\top Z^\top Z\mathbf{w}] \end{aligned}$$

$$= \mathbf{y}^\top \mathbf{y} - 2\mathbf{y}^\top \mathbb{E}[Z] \mathbf{w} + \mathbf{w}^\top \mathbb{E}[Z^\top Z] \mathbf{w}$$

Since $Z = R \circ X$, we have $\mathbb{E}[Z] = \mu X$.

So:

$$\mathbf{y}^\top \mathbb{E}[Z] \mathbf{w} = \mu \mathbf{y}^\top X \mathbf{w}$$

Now, compute $\mathbb{E}[Z^\top Z]$. Let's look at entry (j, k) of $Z^\top Z$:

$$[Z^\top Z]_{jk} = \sum_i Z_{ij} Z_{ik} = \sum_i R_{ij} R_{ik} X_{ij} X_{ik}$$

Now take expectation:

$$\mathbb{E}[Z^\top Z]_{jk} = \sum_i \mathbb{E}[R_{ij} R_{ik}] X_{ij} X_{ik}$$

Since R_{ij} and R_{ik} are independent for $j \neq k$:

$$\mathbb{E}[R_{ij} R_{ik}] = \begin{cases} \mu^2 & j \neq k \\ \mu^2 + \sigma^2 & j = k \end{cases}$$

Therefore:

$$\mathbb{E}[Z^\top Z] = \mu^2 X^\top X + \sigma^2 \text{diag}(X^\top X)$$

We now have:

$$\mathcal{L}(\mathbf{w}) = \mathbf{y}^\top \mathbf{y} - 2\mu \mathbf{y}^\top X \mathbf{w} + \mathbf{w}^\top (\mu^2 X^\top X + \sigma^2 \text{diag}(X^\top X)) \mathbf{w}$$

Group terms:

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{y} - \mu X \mathbf{w}\|_2^2 + \sigma^2 \|\Gamma \mathbf{w}\|_2^2$$

where $\Gamma = (\text{diag}(X^\top X))^{1/2}$.

• (A) is $\boxed{\mu}$

• (B) is $\boxed{\sigma^2}$

Problem 6

Let n be the number of workers and p be total number of model parameters (assumed divisible by n) Each of the n workers sends its gradient (size p) to each of the other $n - 1$ workers.

- Number of messages: $n(n - 1)$
- Size of each message: p

Each of the n workers sends one message of size p to the server, and receives one back.

Type	Number of Messages Sent	Size of Each Message
All-to-All	$n(n - 1)$	p
Parameter Server	$2n$	p
Ring All-Reduce	$2n(n - 1)$	p/n

- Number of messages: $2n$
- Size of each message: p

Each of the n workers splits its gradient into n chunks of size p/n . Then, each worker sends and receives $n - 1$ messages of size p/n . Then in the all-gather phase, this is repeated.

- Number of messages: $n \cdot 2(n - 1) = 2n(n - 1)$
- Size of each message: p/n

Problem 7

(1a)

ResNet-152 has a total of 58,164,298 trainable parameters. The size of the model in memory is approximately

$$\text{Size} = \frac{\sum \# \text{elements} \times \text{element size (bytes)}}{10^6} = \frac{232.657192 \times 10^6 \text{ bytes}}{10^6} = \boxed{232.66 \text{ MB}}$$

(1b)

We are using an NVIDIA Tesla T4. It has a total of:

$$\boxed{15360 \text{ MiB} = 15 \text{ GiB}}$$

(1c)

The total GPU memory usage after loading the model is 346 MiB. Given that the model size is approximately 232.66 MB (or roughly 222 MiB), we estimate the overhead of the CUDA context as $346 \text{ MiB} - 222 \text{ MiB} = \boxed{124 \text{ MiB}}$

(2a)

Based on the memory profiling logs, the GPU memory utilization (in MB) during training with each optimizer is:

- SGD: 2182 MB
- SGD with momentum: 2264 MB
- Adam: 2428 MB

(2b)

The optimizer that consumes the most memory is Adam(2428 MB).

This is expected because the Adam optimizer maintains more internal state per parameter compared to SGD and SGD with momentum. SGD stores only the parameters and their gradients. SGD with momentum adds a velocity term per parameter (one additional tensor per weight). Adam stores both the first and second moment estimates (m_t and v_t) for each parameter, resulting in two additional tensors per parameter.

Therefore, Adam uses the most memory as it requires maintaining multiple moving averages for each model weight.

(3a)

The memory usage for different batch sizes was as follows:

Batch Size	Memory Usage (MB)
4	636.0
16	968.0
64	2264.0
256	8826.0

The largest batch size that was successfully trained was 256. An attempt to train with a batch size of 1024 resulted in a `CUDA OutOfMemoryError`, as the GPU could not allocate the additional memory required.

(3b)

From the training accuracy vs. wall clock time plot, the batch size that yielded the highest training accuracy at the end of 10 epochs was 16, achieving nearly 59% accuracy.

(3c)

The batch size that completed 10 epochs in the least amount of wall clock time was 256.

(3d)

